

GIRAFFE BAKERY WAR

RAPPORT TECHNIQUE

Projet POO - Jeu de Stratégie au Tour par Tour

Développé en Java avec LibGDX

GROUPE : BENZEMRANE Sara ,ARBANE Katia

YOUSFI Asma , EMBAREK Rayane Fares Anis

DURÉE : 1 mois

OUTILS : Visual Studio Code, LibGDX, Tiled, Git/GitHub, Gradle

1. PRÉSENTATION DU PROJET :

CONCEPT :

Jeu de stratégie au tour par tour où le joueur commande des girafes armées d'ustensiles de pâtisserie contre des vagues d'ennemis.

OBJECTIF PÉDAGOGIQUE :

Appliquer les principes de POO (encapsulation, héritage, polymorphisme, abstraction) dans un projet complexe de développement de jeu vidéo.

CARACTÉRISTIQUES PRINCIPALES :

- Combat tactique au tour par tour sur grille
- 3 types d'unités (Whisk, Pan, Rolling Pin Giraffe)
- 4 types de bâtiments productifs (Farm, Mine, Sawmill, Command Center)
- Système de ressources (Gold, Wood, Food, Stone)
- Intelligence artificielle ennemie
- Carte 40×70 tuiles créée avec Tiled (64×64 pixels par tuile)

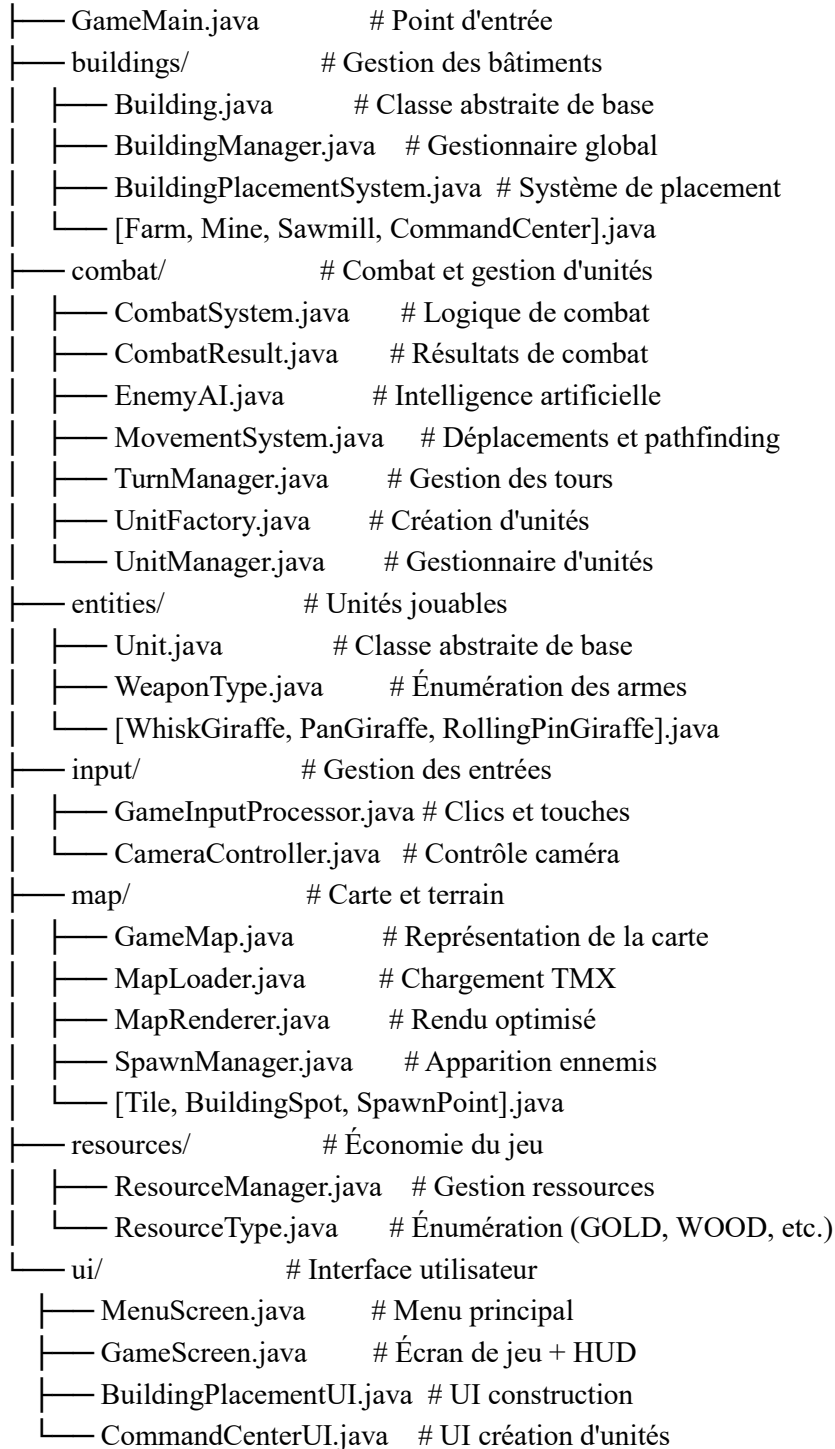
TECHNOLOGIES :

- Langage : Java
- Framework : LibGDX (jeu 2D cross-platform)
- Éditeur de carte : Tiled Map Editor
- Assets : Itch.io, Pixel Art Village, Adobe (128×128 pixels)
- Build : Gradle
- Versionnage : Git/GitHub

2. ARCHITECTURE LOGICIELLE

ORGANISATION EN PACKAGES :

pastrydad.com/



PATTERNS DE CONCEPTION UTILISÉS :

- Abstract Factory : UnitFactory (création avec validation)
- Template Method : Building et Unit (méthodes abstraites)
- Strategy : EnemyAI (comportements agressif/défensif)
- Singleton (implicite) : Managers centralisés
- Composite : GameScreen compose plusieurs managers

FLUX DE DONNÉES :

1. GameMain → MenuScreen (musique, décorations)
2. Clic "NEW GAME" → GameScreen
3. GameScreen initialise :
 - MapLoader charge game_map.tmx
 - ResourceManager (500 Gold, 400 Wood, 300 Food, 200 Stone)
 - UnitManager, BuildingManager, CombatSystem, etc.
4. Boucle de jeu (60 FPS) :
 - Input : GameInputProcessor → Actions
 - Update : TurnManager gère alternance joueur/IA
 - Render : Map → Buildings → Units → UI

3. SYSTÈMES PRINCIPAUX

3.1 SYSTÈME DE CARTE

CHARGEMENT (MapLoader.java) :

- Parse fichier TMX
- Extraction multi-layers (terrain, décorations, overlay)
- Conversion coordonnées Tiled (top-left) → LibGDX (bottom-left)
- Chargement BuildingSpots et SpawnPoints

RENDU (MapRenderer.java) :

- Culling viewport : seules tuiles visibles rendues
- Performance optimisée (~100-200 tuiles au lieu de 2800)

STRUCTURE DE DONNÉES :

List<Tile>[][] tiles // Grille 2D avec layers multiples

3.2 SYSTÈME DE COMBAT

HIÉRARCHIE DES UNITÉS :

- Unit (abstraite) : hp, attack, defense, range, moveSpeed
- WhiskGiraffe : Équilibrée (60 HP, attaque 12, portée 1, vitesse 3)
 - Spécial : Double frappe

- PanGiraffe : Tank (120 HP, attaque 20, portée 1, vitesse 2)
Spécial : Coup lourd 150%
- RollingPinGiraffe : Distance (50 HP, attaque 18, portée 4, vitesse 3)
Spécial : Tir précis (ignore défense)

CALCUL DES DÉGÂTS :

```
baseDamage = attacker.attack - (defender.defense / 2)
finalDamage = baseDamage * randomFactor(0.8-1.2)
if (random < 5%) then CRITIQUE : finalDamage *= 2
finalDamage += weaponBonus
finalDamage = max(1, finalDamage)
```

DÉROULEMENT :

CombatSystem.executeCombat() → Validation → Calcul → Application → Logging

3.3 INTELLIGENCE ARTIFICIELLE

ALGORITHME (EnemyAI.java) :

Pour chaque ennemi vivant :

1. target = findBestTarget()
Score = proximité + HP_faibles + dangerosité + bonus_kill
2. Si à portée: alors attaquer
3. Sinon : se déplacer vers cible
Puis attaquer si maintenant à portée

STRATÉGIE :

- Priorité aux cibles blessées (finir les kills)
- Bonus énorme si kill garanti (+100 points)
- Facteur d'agressivité ajustable (0.0-1.0)

3.4 SYSTÈME DE MOUVEMENT

MovementSystem.java :

- getReachablePositions() : Cases accessibles (Manhattan distance)
- canMoveToPosition() : Validation (walkable, libre, distance OK)
- findPath() : Pathfinding BFS (Breadth-First Search)

3.5 SYSTÈME DE BÂTIMENTS

HIÉRARCHIE :

- Building (abstraite) : cost, buildTime, position, texture
- Farm : +15 Food/tour (coût : 40 Wood)

- Mine : +10 Stone/tour (coût : 50 Wood)
- Sawmill : +12 Wood/tour (coût : 30 Gold)
- CommandCenter : Création d'unités (coût : 100 Wood + 100 Stone)

CONSTRUCTION :

buildStep() appelé chaque tour → remainingBuildTime--

Si terminé → constructed = true → onConstructionComplete()

PRODUCTION :

TurnManager.endPlayerTurn() → BuildingManager.processTurn()

→ Chaque bâtiment construit produit ses ressources

3.6 SYSTÈME DE RESSOURCES

ResourceManager.java :

- Map<ResourceType, Integer> resources
- add(type, amount) : Ajout de ressources
- canAfford(cost) : Vérification disponibilité
- spend(cost) : Dépense avec validation

Ressources de départ :

- 500 Gold, 400 Wood, 300 Food, 200 Stone

3.7 SYSTÈME DE TOURS

TurnManager.java :

1. TOUR JOUEUR

- Joueur déplace/attaque
- Appui sur T

2. FIN TOUR JOUEUR

- BuildingManager.processTurn() → Production ressources
- unitManager.resetPlayerUnits()

3. TOUR ENNEMI

- SpawnManager.onEnemyTurn() → Nouveaux ennemis
- EnemyAI.executeTurn() → IA joue
- unitManager.cleanupDeadUnits() → Score +50/kill

4. NOUVEAU TOUR JOUEUR

- turnNumber++

4. INTERFACE UTILISATEUR

4.1 MENUSCREEN

- Titre "GIRAFFE BAKERY WAR" (couleurs variées)
- Boutons : NEW GAME (rose), QUIT (jaune)
- 15 sprites de girafes animés (flottement)
- Musique : assets/music/menu.mp3

4.2 GAMESCREEN HUD

Affichage permanent :

- Ressources (Gold, Wood, Food, Stone)
- Score (+50 par ennemi tué)
- Tour actuel
- Stats unité sélectionnée

Overlays visuels :

- Cases VERTES : Déplacement possible
- Cases ROUGES : Attaque possible
- Cercle jaune : Unité sélectionnée

4.3 BUILDINGPLACEMENTUI

Panneau latéral (300×500px) :

- 3 boutons : Farm [F], Mine [M], Sawmill [S]
- Affichage coûts avec code couleur (vert/rouge)
- Aperçu placement (rectangle vert/rouge)
- Raccourcis : B (toggle), F/M/S (sélection), ESC (annuler)

4.4 COMMANDCENTERUI

Création d'unités :

- 3 boutons : Whisk [1], Pan [2], Rolling Pin [3]
- Affichage coûts
- Raccourcis : C (toggle), 1/2/3 (sélection)

5. GESTION DES ENTRÉES

GAMEINPUTPROCESSOR :

Priorité de traitement :

1. Caméra (clic droit) → CameraController
2. UI (si visible) → BuildingPlacementUI / CommandCenterUI
3. Placement mode → BuildingPlacementSystem
4. Unités → Sélection / Déplacement / Attaque

FIX CRITIQUE - screenToTile() :

```
camera.update(); // ← ESSENTIEL (bug décalage sélection)
Vector3 worldPos = camera.unproject(screenPos);
int tileX = (int)(worldPos.x / tileWidth);
```

CAMERACONTROLLER :

- WASD / Flèches : Déplacement (300 px/s)
- Molette : Zoom (limité 0.5-3.0)
- Clic droit + glisser : Pan
- R : Recentrage
- Clamping aux limites de carte

6. PROBLÈMES ET SOLUTIONS

PROBLÈME 1 : FULLSCREEN BUG

Symptômes : Tuiles décalées, rendu incorrect

Tentatives : Ajustement viewport, recalcul projections

Solution : Désactivation fullscreen → Mode fenêtré uniquement

PROBLÈME 2 : DÉCALAGE SÉLECTION (2 tuiles à gauche)

Cause : Conversion screen→world sans camera.update()

Solution : Ajout de camera.update() avant unproject()

```
private Vector2 screenToTile(int screenX, int screenY) {
    camera.update(); // ← FIX
    ...
}
```

PROBLÈME 3 : ZOOM OUT EXCESSIF

Symptômes : Carte disparaît, bordures noires

Solution : Limites zoom (min: 0.5, max: 3.0) + clamping

PROBLÈME 4 : COORDONNÉES TILED → LIBGDX

Cause : Origines différentes (top-left vs bottom-left)

Solution : Flip vertical Y

```
double flippedY = mapPixelHeight - tiledY - height;
```

7. MÉTRIQUES DU PROJET

CODE :

- 40 classes Java
- ~5000-6000 lignes de code
- 8 packages modulaires

ASSETS :

- 7 sprites (3 unités + 4 bâtiments) : 128×128 px
- ~10 tilesets terrain : 64×64 px par tuile
- 1 carte TMX : 40×70 tuiles = 2800 tuiles
- 1 fichier musique (menu.mp3)

FONCTIONNALITÉS :

- Combat au tour par tour
- 3 types d'unités avec compétences
- 4 types de bâtiments productifs
- IA ennemie tactique
- Gestion 4 ressources
- Système de spawn dynamique
- Interface complète (HUD, menus)
- Contrôle caméra (zoom, pan)
- Score et conditions victoire/défaite

8. CONCLUSION**OBJECTIFS ATTEINTS :**

- Application POO (héritage, polymorphisme, abstraction, encapsulation)
- Architecture modulaire avec séparation des responsabilités
- Patterns de conception (Factory, Template Method, Strategy)
- Jeu fonctionnel et jouable
- Travail collaboratif avec Git/GitHub

COMPÉTENCES ACQUISES :

- Maîtrise LibGDX et développement jeux 2D
- Algorithmes (pathfinding BFS, IA tactique)
- Gestion systèmes de coordonnées
- Intégration Tiled Map Editor
- Résolution bugs complexes
- Versionnage et collaboration

LIMITATIONS :

- Pas de fullscreen stable
- Une seule carte
- Pas de sauvegarde/chargement
- IA basique
- Animations limitées

AMÉLIORATIONS FUTURES :

- Correction bug fullscreen

- Nouvelles cartes et unités
- Système de sauvegarde
- Mode campagne
- Amélioration IA
- Animations et effets sonores
- Multijoueur local

BILAN :

GIRAFFE BAKERY WAR démontre une bonne maîtrise de la POO et du développement de jeux vidéo. L'architecture modulaire facilite l'extensibilité. Les défis techniques (coordonnées, caméra, pathfinding) ont été des opportunités d'apprentissage précieuses. Le projet constitue une base solide pour de futures améliorations.

FIN DU RAPPORT TECHNIQUE**ANNEXES :**

- Manuel utilisateur
- Code source : [URL GITHUB À COMPLÉTER]
- Rapport technique
- Cahier des charges

GIRAFFE BAKERY WAR TEAM - Projet POO